ERROS ASSÍNCRONOS E DIAGNÓSTICO

Lista de exercícios — Async, ErrorHandler, RequestId, Watch e Memória

Terceira lista da sequência de técnicas de debugging em Node.js e TypeScript

Atividade prática

Esta lista contém cinco exercícios. Os códigos possuem falhas intencionais e os trechos de apoio não entregam a resolução completa. O tema comum é o defeito que não aparece na linha onde foi escrito: ele aparece um tique depois, em outro callback, em outra requisição ou somente depois de milhares de execuções.

Exercícios

- 7 - Debug de erros assíncronos com try/catch, promises e setTimeout.
- 8 - ErrorHandler global no Express com resposta padronizada.
- 9 - Logs com identificador de requisição.
- 10 - Observação de estado com Watch Expressions no VS Code.
- 11 - Debug de memory leak simples com process.memoryUsage().

Em cada exercício, registre o sintoma, sua hipótese, a evidência encontrada e a correção aplicada. Aqui a evidência costuma ser uma sequência: a ordem em que as linhas apareceram importa tanto quanto o conteúdo delas.

Pré-requisitos

```
npm install express uuid
npm install -D typescript ts-node @types/node @types/express

# o Node 18+ já traz fetch e crypto.randomUUID nativos
node --version
```

Reaproveite o projeto das listas anteriores. O logger do exercício 4 e o launch.json do exercício 3 são usados novamente aqui.

Orientações de investigação

Nenhum destes defeitos é resolvido lendo o código com mais atenção. Todos exigem provocar o cenário e observar o que a máquina faz, na ordem em que faz.

Etapa	O que fazer
1. Reproduzir	Provocar o caminho de falha de propósito, não esperar que ele aconteça sozinho.
2. Observar	Registrar a saída completa e a ordem das linhas, inclusive as que não apareceram.
3. Formular hipótese	Apontar em qual momento do ciclo de execução a falha escapou.
4. Investigar	Usar breakpoint, watch, medição de memória ou filtro de log por identificador.
5. Corrigir	Aplicar a menor mudança que resolva a causa.
6. Verificar	Repetir o cenário e comparar com a medição feita antes da correção.

Registro obrigatório

```
Sintoma:
Passos para reproduzir:
Resultado esperado:
Resultado obtido (saída copiada, na ordem em que apareceu):
Hipótese:
Ferramenta utilizada:
Causa encontrada:
Como a correção foi validada:
```

Regras

- Não usar any no catch nem no tipo do erro.
- Não deixar catch vazio nem capturar o erro apenas para devolver sucesso.
- Não expor stack trace, caminho de arquivo ou mensagem interna ao usuário final.
- Não remover o log de erro para a saída dos testes ficar limpa.
- Não tratar sintoma de memória aumentando o limite do heap.
- Remover instrumentação temporária depois da investigação.

Avaliação: o raciocínio usado para encontrar a falha vale tanto quanto o resultado final.

Exercício 7 - Debug de Erros Assíncronos

A função deveria carregar um arquivo de configuração e registrar qualquer falha. Ao apontar para um arquivo inexistente, o bloco catch não é acionado, um aviso estranho aparece no terminal e, poucos instantes depois, o processo morre.

```
import fs from 'fs';

function readConfig(path: string) {
  try {
    fs.promises.readFile(path, 'utf-8').then((content) => {
      console.log('config carregada', JSON.parse(content));
    });
  } catch (error) {
    console.error('falha ao ler config', error.message);
  }
}

function retryLater() {
  setTimeout(() => {
    throw new Error('tentativa de releitura falhou');
  }, 200);
}

readConfig('./config/app.json');
retryLater();
```

Tarefas

- Executar com um arquivo inexistente e copiar a saída completa do terminal.
- Explicar por que o catch não captura a falha de leitura.
- Explicar por que o erro lançado dentro do setTimeout derruba o processo.
- Reescrever a função com async/await para que a falha chegue ao catch.
- Tipar o valor capturado como unknown e estreitar o tipo antes de usar.
- Diferenciar o erro esperado (arquivo ausente) do inesperado (conteúdo inválido).
- Imprimir o erro com console.error preservando o stack.
- Repetir o exercício usando fetch contra uma URL inválida e contra uma rota 404.

Pistas graduais

- Descubra em que momento o try já terminou de executar em relação à conclusão da promise.
- Procure o código de erro que o Node atribui a um arquivo inexistente.
- Compare console.error(err.message) com console.error(err) e veja o que se perde.
- Investigue o evento unhandledRejection e o que o Node faz com ele por padrão.
- Verifique se fetch rejeita a promise quando a resposta tem status 404.

Exercício 7 - Apoio e verificação

Trecho de apoio

```
async function readConfig(path: string): Promise</* o que a função devolve? */> {
  try {
    const content = /* aguarde a leitura aqui dentro */;
    return JSON.parse(content);
  } catch (error: unknown) {
    if (/* como identificar arquivo inexistente? */) {
      throw new AppError('configuração não encontrada', 404);
    }
    /* registre o inesperado e decida se repassa ou converte */
  }
}

process.on(/* qual evento avisa sobre rejeição não tratada? */, (reason) => {
  /* registre e decida o destino do processo */
});
```

Cenários mínimos

Cenário	O que verificar
Arquivo existente e válido	Retorno correto e nenhuma linha de erro.
Arquivo inexistente	O catch é acionado e o código do erro é identificado.
Arquivo com JSON inválido	A falha de parse não é confundida com arquivo ausente.
fetch em host inexistente	Erro de rede tratado, processo continua vivo.
fetch com resposta 404	A promise não rejeita, o status precisa ser checado.

Perguntas orientadoras

- Por que um try/catch síncrono não alcança o erro de uma promise sem await?
- Qual a diferença entre uma falha de rede e uma resposta HTTP de erro?
- Por que console.error e não console.log para registrar falhas?
- O que o Node faz por padrão com uma rejeição não tratada?
- Por que o stack de um erro assíncrono costuma ser menos informativo, e como melhorar isso?

Critério: nenhum dos cinco cenários pode terminar com o processo caindo ou com aviso de rejeição não tratada.

Exercício 8 - Criando um ErrorHandler Global

O projeto já possui um AppError com statusCode, mas toda falha chega ao cliente como 500 com texto puro. Em algumas rotas o erro nem chega ao handler: a requisição simplesmente fica pendurada até o timeout.

```
import express, { Request, Response } from 'express';

const app = express();

app.get('/usuarios/:id', async (req, res) => {
  const user = await findUser(req.params.id);

  if (!user) {
    throw new AppError('usuário não encontrado', 404);
  }

  res.json(user);
});

function errorHandler(err: any, req: Request, res: Response) {
  console.log(err);
  res.status(500).send(err.message);
}

app.use(errorHandler);
```

Tarefas

- Chamar a rota com um id inexistente e registrar status e corpo da resposta.
- Explicar por que o Express não reconhece a função como middleware de erro.
- Tipar o handler com ErrorRequestHandler em vez de any.
- Respeitar o statusCode do AppError em vez de responder 500 sempre.
- Padronizar a resposta em JSON com statusCode, message e timestamp.
- Garantir que a mensagem de um erro inesperado não chegue ao cliente.
- Fazer o erro lançado dentro de uma rota async chegar até o handler.
- Registrar o erro com o logger do exercício 4, incluindo o stack.

Exercício 8 - Apoio e verificação

Trecho de apoio

```
import { ErrorRequestHandler } from 'express';

export const errorHandler: ErrorRequestHandler = (err, req, res, next) => {
  if (/* a resposta já foi iniciada? */) {
    return next(err);
  }

  if (/* é um erro conhecido da aplicação? */) {
    return res.status(/* ... */).json({
      statusCode: /* ... */,
      message: /* ... */,
      timestamp: /* formato padronizado */
    });
  }

  /* registre o erro inesperado com stack, sem devolvê-lo */
  return res.status(500).json({ /* corpo genérico com os mesmos campos */ });
};
```

Pistas graduais

- Conte os parâmetros da função e compare com a assinatura que o Express espera.
- Verifique a posição do app.use(errorHandler) em relação às rotas.
- Descubra o que acontece com uma exceção lançada dentro de um handler async no Express 4.
- Investigue res.headersSent e o erro de cabeçalho enviado duas vezes.
- Compare o corpo devolvido por res.send(string) com o de res.json(objeto).

Cenários mínimos

Requisição	O que verificar
Usuário existente	Status 200 e corpo do usuário, sem passar pelo handler.
Usuário inexistente	Status 404 e JSON padronizado com os três campos.
Erro inesperado na camada de dados	Status 500, mensagem genérica e stack apenas no log.
Rota async que rejeita	O handler é alcançado e a requisição não fica pendurada.
Rota não registrada	Resposta 404 no mesmo formato das demais.

Perguntas orientadoras

- Por que o middleware de erro precisa declarar quatro parâmetros?
- O que acontece quando o servidor tenta responder duas vezes à mesma requisição?
- Qual informação do erro pertence à resposta e qual pertence ao log?
- Que vantagem um corpo de erro padronizado traz para quem consome a API?

Critério: todas as falhas devem sair no mesmo formato, com o status correto e sem detalhe interno no corpo da resposta.

Exercício 9 - Logs com Identificador de Requisição

Sob carga, o log vira uma lista de linhas sem dono: não dá para saber quais mensagens pertencem à mesma requisição. A tentativa abaixo gera um identificador, mas ele não acompanha o resto da execução.

```
let requestId = 0;

app.use((req, res, next) => {
  requestId++;
  logger.info('requisição recebida', { requestId });
  next();
});

app.get('/pedidos/:id', async (req, res) => {
  logger.info('buscando pedido');
  const pedido = await buscarPedido(req.params.id);
  logger.info('pedido encontrado');
  res.json(pedido);
});
```

Tarefas

- Disparar três requisições simultâneas e mostrar por que o log atual não permite separá-las.
- Trocar o contador por um UUID gerado a cada requisição.
- Reaproveitar o identificador recebido do cliente quando ele existir.
- Devolver o identificador em um cabeçalho da resposta.
- Fazer o identificador aparecer em todas as linhas da requisição, inclusive nas de erro.
- Propagar o valor até as funções de serviço sem passá-lo como parâmetro em toda a cadeia.
- Declarar o campo adicional em Request sem recorrer a any.
- Filtrar o log por um identificador específico e mostrar a sequência completa.

Trecho de apoio

```
import { randomUUID } from 'node:crypto';
import { AsyncLocalStorage } from 'node:async_hooks';

const storage = new AsyncLocalStorage</* qual tipo guarda o contexto? */>();

app.use((req, res, next) => {
  const id = /* header recebido do cliente ou um novo identificador */;
  res.setHeader(/* qual cabeçalho devolve o id? */, id);
  storage.run(/* contexto */, () => next());
});

export function logInfo(message: string, meta = {}) {
  logger.info(message, { ...meta, requestId: /* leia do contexto atual */ });
}
```

Exercício 9 - Investigação e verificação

Pistas graduais

- Pense no que acontece com o contador quando o processo reinicia ou quando existem dois processos.
- Verifique se o valor gerado no middleware está visível dentro do handler da rota.
- Investigue o child logger da biblioteca escolhida e o que ele acrescenta automaticamente.
- Estude AsyncLocalStorage e por que ele sobrevive a um await.
- Descubra qual cabeçalho é convenção para transportar esse identificador entre serviços.

Cenários mínimos

Cenário	O que verificar
Três requisições simultâneas	Cada grupo de linhas tem um identificador distinto e estável.
Cliente envia o próprio identificador	O valor recebido é reaproveitado, não substituído.
Rota que falha	A linha de erro carrega o mesmo identificador das linhas anteriores.
Dois processos sob PM2	Não há colisão de identificadores entre os processos.
Filtro por um identificador	A sequência recuperada cobre do início ao fim da requisição.

Perguntas orientadoras

- Por que um contador incremental não serve em um sistema com vários processos?
- Qual a diferença entre repassar o identificador por parâmetro e mantê-lo em contexto?
- O que acontece com o contexto quando a execução atravessa um await?
- Como esse identificador ajuda a seguir uma chamada que passa por vários serviços?
- Que custo essa instrumentação acrescenta e quando ele se justifica?

Entrega: além do código, anexe o log de três requisições concorrentes e o mesmo log filtrado por um único identificador.

Exercício 10 - Observando Comportamento com Watch

A função deveria somar os itens e aplicar um desconto quando o pedido ultrapassa R$ 200,00. O total devolvido muda conforme a ordem dos itens da lista, mesmo com os mesmos produtos e as mesmas quantidades.

```
interface Item {
  price: number;
  qty: number;
}

function applyDiscount(items: Item[]): number {
  let total = 0;
  let applied = 0;

  for (let index = 0; index < items.length; index++) {
    total += items[index].price * items[index].qty;

    if (total > 200 && applied < 2) {
      total = total - total * 0.05;
      applied++;
    }
  }

  return Number(total.toFixed(2));
}

console.log(applyDiscount([
  { price: 100, qty: 1 },
  { price: 150, qty: 1 },
  { price: 50, qty: 2 }
]));
```

Tarefas

- Executar a função e registrar o resultado.
- Executar de novo com os mesmos itens em ordem diferente e comparar os dois totais.
- Colocar um breakpoint dentro do for, na linha da condição.
- Adicionar ao painel Watch: total, applied, index e items[index].price * items[index].qty.
- Acrescentar ao Watch uma expressão com a soma bruta esperada, para comparar a cada parada.
- Usar Step Over e anotar os valores em pelo menos três iterações.
- Explicar em qual iteração o total deixa de corresponder ao valor correto.
- Corrigir e comprovar que a ordem dos itens não altera mais o resultado.

Pistas graduais

- Compare, a cada parada, o valor de total com a soma dos itens já percorridos.
- Observe sobre qual valor o desconto está incidindo.
- Verifique quantas vezes o desconto é aplicado em uma única execução.
- Pense se o momento de aplicar o desconto deveria ser dentro ou fora do laço.
- Teste um breakpoint condicional para parar somente quando applied mudar.

Exercício 10 - Evidências e perguntas

Cenários mínimos

Entrada	O que verificar
Itens somando R$ 180,00	Nenhum desconto aplicado.
Itens somando exatamente R$ 200,00	O limite é tratado conforme a regra escrita.
Mesmos itens em ordem invertida	O total precisa ser idêntico ao da ordem original.
Item único de R$ 500,00	Desconto aplicado uma única vez, sobre o valor correto.
Lista vazia	Retorno definido, sem erro.

Evidências solicitadas

- Descrição do breakpoint utilizado e da condição, se houver.
- Conteúdo do painel Watch em pelo menos três iterações.
- Os dois totais obtidos com a mesma lista em ordens diferentes.
- Explicação do ponto exato em que o valor se desvia do esperado.
- Resultados dos cinco cenários após a correção.

Perguntas para responder

- Qual a diferença entre o painel Watch e o painel de variáveis locais?
- Uma expressão colocada no Watch pode alterar o estado do programa?
- Quando um breakpoint condicional é mais útil que um breakpoint comum?
- O que é um logpoint e em que situação ele substitui um console.log?
- Por que observar a mudança de uma variável é mais revelador que ler o valor final?

Exercício 11 - Debug de Memory Leak Simples

O serviço funciona bem nos primeiros minutos e vai ficando lento até o processo ser encerrado pelo sistema. O trecho abaixo reproduz o comportamento de forma artificial e concentrada.

```
const cache: Record<string, Buffer> = {};
const listeners: Array<() => void> = [];

function handleRequest(id: string) {
  cache[id] = Buffer.alloc(1024 * 512);
  listeners.push(() => console.log(id));

  setInterval(() => {
    void cache[id];
  }, 1000);
}

for (let i = 0; i < 2000; i++) {
  handleRequest(`req-${i}`);

  if (i % 200 === 0) {
    console.log(process.memoryUsage());
  }
}
```

Tarefas

- Executar e registrar os valores impressos a cada 200 iterações.
- Formatar a medição em megabytes, em vez de bytes, para facilitar a comparação.
- Identificar qual das estruturas cresce sem limite e por que nada é liberado.
- Explicar o papel do temporizador periódico na retenção dos dados.
- Reduzir o limite de heap na execução para provocar o estouro mais cedo.
- Aplicar limite de tamanho ou expiração no cache e cancelar o temporizador.
- Comparar a curva de consumo antes e depois da correção, com os mesmos pontos de medição.
- Como desafio, capturar dois heap snapshots com --inspect e comparar a diferença.

Trecho de apoio

```
function snapshot(label: string) {
  const { rss, heapTotal, heapUsed, external } = process.memoryUsage();
  const mb = (value: number) => /* converta e arredonde */;

  console.log(label, { rss: mb(rss), heapTotal: mb(heapTotal), heapUsed: mb(heapUsed) });
}

// execução com limite reduzido e coleta manual habilitada
// node --max-old-space-size=/* valor baixo */ --expose-gc dist/leak.js
```

Exercício 11 - Evidências e perguntas

Pistas graduais

- Descubra a diferença entre rss, heapTotal, heapUsed e external.
- Observe o que muda na curva quando o temporizador periódico é removido.
- Pense no que a função passada ao temporizador mantém viva mesmo sem ser chamada.
- Force uma coleta de lixo e verifique se o heapUsed volta ao patamar anterior.
- Compare uma estrutura que segura referências fortes com uma que não segura.

Cenários mínimos

Execução	O que verificar
200 iterações	Consumo inicial e ponto de partida da medição.
2000 iterações	Crescimento contínuo do heapUsed entre as medições.
Sem o temporizador periódico	Mudança na curva e no que a coleta consegue liberar.
Com limite no cache	Consumo estabiliza em um patamar previsível.
Com heap reduzido	Momento em que o processo é encerrado por falta de memória.

Perguntas para responder

- Por que uma única medição não é suficiente para afirmar que existe vazamento?
- Qual a diferença entre consumo alto e vazamento de memória?
- Por que temporizadores e coleções de callbacks são causa frequente desse problema?
- O que a comparação entre dois heap snapshots revela que a medição isolada não revela?
- Em que situação uma estrutura com referências fracas resolve o caso?

Checklist final

Exercício	Entrega mínima
7	Erro assíncrono capturado, catch tipado como unknown, cinco cenários testados e nenhuma rejeição não tratada.
8	ErrorHandler tipado, statusCode respeitado, JSON padronizado e rota async alcançando o handler.
9	Identificador único por requisição em todas as linhas, cabeçalho de resposta e log filtrado por id.
10	Breakpoint, painel Watch em três iterações, causa explicada e resultado estável entre ordens diferentes.
11	Medições antes e depois, estrutura vazando identificada, correção aplicada e curva estabilizada.

Entrega final: envie o código corrigido e um pequeno registro explicando como cada ferramenta levou até a causa do problema. Para esta lista, anexe as saídas que serviram de evidência na ordem em que foram observadas, antes e depois de cada correção.